

COMP3050 Team Project Assignment 2026 – API v3

Overview

The project involves building a server which tracks the state of a simple 2D tile-based virtual world. This virtual world could be used for playing a simple adventure game.

Students create and maintain the game world in a server, which communicates with a web-based client using a web API. This document specifies the API version number v3. (The API version number has increased because there are new APIs specified in this document. Also note that the v1 and v2 APIs may have also changed in v3 to support new features.)

Note: Students don't create the client or the tile images, which will all be supplied to the students. This is to avoid students getting distracted by the game play elements, because the intention is to focus on the server-side deployment and maintenance.

Game play

An example of game play might look like this:



Map

On the client window, the map view is shown with the player's character in the central location.

The player's view has a fixed size: an 11x11 square grid. But the world map may be larger.

The map has several elements:

- A character which can walk around within the map. A player will see the map with their own character positioned at the central tile.
- There can be multiple players logged in at the same time. Players can see each other's characters as they walk around the shared map.
- Surfaces, which player characters can walk on, such as grass, dirt, and various kinds of flooring.
- Impassable elements, such as walls, doors, and water.
- Items which can be taken and placed elsewhere.

Movement

Characters can be moved one space in one of the four cardinal directions:

Direction	Direction name	dy	dx	Description	Client key
N	North	-1	0	One space north.	W or ↑
W	West	0	-1	One space west.	A or ←
S	South	+1	0	One space south.	S or ↓
E	East	0	+1	One space east.	D or →

Diagonal motion and moving more than one space are disallowed.

Adjacency

The term *adjacent* refers to a map location that is horizontally or vertically one square away from the player's character. Diagonals are not considered adjacent.

Movement is only allowed to adjacent spaces.

Interacting with nearby map elements is also only allowed for adjacent spaces. For example, to open a door, the player's character must be standing directly North, West, South, or East of the door to interact with it.

API

The API supports several operations:

/login	POST: name= <i>NAME</i> encpswrd= <i>encrypted_password</i>	Player name and encrypted player game password must be sent in the message body via a POST to the server. The front end client software will encrypt the player game password before sending it to the server. • Returns 200 if the name and password are correct, with a JSON body containing a session token to be used for all further commands. • Returns 400 (Bad Request) if the name and/or password weren't sent. • Returns 401 (Unauthorized) if the name and/or encrypted password sent don't match the server's records.
/logout	?session= <i>SESSION</i>	The server removes its server-side record of the session token, and that token can't be used afterwards. • Returns 200 (OK) after ending the current gaming session for the player identified by the session token. • Returns 401 (Unauthorized) if the session token is invalid or does not identify a currently active player (e.g. if the player has already signed out).
/move	?dy= <i>DY</i> &dx= <i>DX</i> &session= <i>SESSION</i>	Move the character in the corresponding direction if possible. Locations are relative to the character, so dy=0&dx=+1 is East one space, and dy=+1&dx=0 is South one space. • Returns 200 (OK) with confirmation of the new location if successful. • Returns 204 (No Content) if motion is blocked or invalid (e.g. if moving diagonally or more than one space). • Returns 401 (Unauthorized) if the session token is invalid.
/info	?y= <i>Y</i> &x= <i>X</i> &session= <i>SESSION</i>	Fetch information about a particular space, including floor tile kind. The location given in the y and x parameters are the character's absolute location in the map. The y co-ordinate increases in the South direction, the x co-ordinate increases in the East direction. • Return 200 (OK) with JSON information if successful. • Returns 204 (No Content) if the server is deliberately not sending information about that tile or its surrounds (e.g. if the character is not at the location specified, or if nothing has changed since the last identical request by the same client). • Returns 401 (Unauthorized) if the session token is invalid.
/take	?session= <i>SESSION</i>	Take the item that is at relative location (0,0), that is, where the character is currently located. If the item is of the same class as another item already in the character's inventory, the two items are swapped, so that the inventory item is effectively placed into the same location. • Return 200 (OK) if successful. • Returns 204 (No Content) if the action can't be completed, for example if there isn't any movable item at the current location. • Returns 401 (Unauthorized) if the session token is invalid.

/place	?session= <i>SESSION</i>	Place an item from the player character's inventory on the ground. The item will be placed at the relative location (0,0), that is, where the character is currently located. • Return 200 (OK) if successful. • Returns 204 (No Content) if an item can't be placed, for example if there is nothing in the character's inventory. • Returns 401 (Unauthorized) if the session token is invalid.
/use	?dy= <i>DY</i> &dx= <i>DX</i> &session= <i>SESSION</i>	Use an adjacent map element, potentially changing the map state. For example, using a door may change its state from closed to open. Locations are relative to the character, so dy=0&dx=-1 is West one space, and dy=-1&dx=0 is North one space. • Return 200 (OK) with JSON information if successful. • Returns 204 (No Content) if no usage could occur, for example if there was no map element at that location which could be used. • Returns 401 (Unauthorized) if the session token is invalid.

LOGIN requests

To authenticate a player with the server, a LOGIN request must be sent.

The LOGIN request must be a POST message containing both a player's character name on that server, and an encrypted player game password (see below). If either or both of these are missing, the request is invalid.

The POST message's body is a textual JSON object.

The player's character name is sent in the `name` field. For now, each name is restricted to ASCII letters and hyphens '-', and are case-sensitive. For example, "Baradun", "NPC-Man", and "willow" are valid names, whereas "Greg the Garlic Farmer", "britt95", and "d&elion" are invalid because they contain invalid symbols, spaces, or numbers.

The password corresponding to a player's character is sent in an encrypted form to the server, within the `encpswrd` field. The client-side software encrypts the password. The server does not need to decrypt the password, but instead compares it with an encrypted value on the server.

The encryption scheme is currently very simple, it is merely a human-readable hexadecimal SHA-256 hash of the string formed from the name followed by a semi-colon ';' followed by the password. For example, if the character name was "Baelin" and the password was "Nice day for fishing." the encrypted password would be the SHA-256 hash of "Baelin;Nice day for fishing.", which is "841eb70b9dd5019642751955afbb960a0f741129877db34ceb420a8fb4a9d1dd".

Warning: Because this is not a strong encryption scheme, it is important for testers of the software to never reuse names or passwords from other services, social media, or educational platforms.

If that player's character is not currently logged in, the server will generate a session token and return it to the client in a JSON message. For example:

```
{
  "session": "K3mdn8Ah2j8s099pqDb3Sz32"
}
```

The form that the session token takes is a case-sensitive string of between 20 to 80 ASCII characters, consisting of only uppercase letters, lowercase letters, and numbers. (A hexadecimal number would satisfy this requirement.) The tokens should be sufficiently unpredictable that different players will not be assigned the same session token. A random number may be used to produce a token. Session tokens are opaque to the client; they contain no fields or components that the client can examine.

The server software maintains a list of valid session tokens. When a player logs in, a session token is generated and recorded by the server (e.g. in a key-value store in memory), and sent to the player's client software. When the player logs out, the session token is revoked by the server, so it is removed from the server's records and can no longer be used by that player or any other.

Session tokens are sent with other requests (e.g. MOVE and INFO) and are looked up by the server to validate and find the corresponding player character information. Invalid or missing tokens result in 401 errors for all other API requests.

LOGOUT requests

When a player logs out, the session token is revoked by the server, so it is removed from the server's records and can no longer be used by that player or any other.

If an invalid session token is sent with the LOGOUT request or with any other API request, a 401 error is produced.

However, if the session token is valid, the LOGOUT request causes the server to invalidate and remove that session token from the server's store of session tokens. Any subsequent use of that session token would result in a 401 error.

Ideally, session tokens are never reused, but this is not a requirement.

MOVE requests

Every MOVE request must be accompanied by a valid session token. For ease of explanation, this token is explained in the LOGIN request section, and will be omitted from MOVE examples below.

The central location where the player's character appears in the view is assigned the number (0,0) in the relative (DY,DX) coordinate system used by MOVE operations. Movement requests for game spaces are relative to character's current location, i.e. relative to that (0,0) view location.

One dy and one dx parameter may be given in each MOVE request. A missing parameter is assumed to be zero. So the following are valid:

Request	Validity	Notes
/move?dy=-1&dx=0	valid	Move N.
/move?dy=-1	valid	Defaults dx=0
/move?dx=+1	valid	Defaults dy=0
/move?dy=0&dx=0	valid	Valid but does nothing. No movement.
/move	valid	Defaults dy=0 and dx=0. No movement.

But the following are invalid:

Request	Validity	Notes
/move?dy=+1&dx=+1	invalid	Diagonal motion is invalid.
/move?dy=-2&dx=0	invalid	Moving more than one space is invalid.

Move requests may fail for various reasons, such as going beyond the edge of the map, or intersecting a blocking map element such as a brick wall or another player's character. In such cases, a 204 status code is returned with an empty response body, and no change is made to the server's recorded location of the player's character.

Valid move requests which succeed cause the server's recorded location of the player's character to be updated to the new location, and a 200 status code to be returned, with a JSON response body reporting the new absolute map x and y location of the player's character. For example:

```
{
  "y": 0,
  "x": 6
}
```

INFO requests

Every INFO request must be accompanied by a valid session token. For ease of explanation, this token is explained in the LOGIN request section, and will be omitted from INFO examples below.

The client uses the INFO request to discover information about locations nearby the character. An INFO request uses absolute (Y,X) locations, relative to the map's co-ordinate system. The y co-ordinate increases in the South direction, the x co-ordinate increases in the East direction.

If an invalid INFO request is made, the server returns a 204 status code, and the message body will be empty. A request is invalid if the requested x and y values don't match the current player character location. A 204 may also be sent if no updates are required, e.g. if nothing has changed.

Upon success, an INFO request returns a 200 status, and the message body will contain JSON data which describes the specified space.

The INFO request's JSON response specifies:

- The "info", a rectangular array (an array of arrays) of tile strings, which the client uses to display the appropriate image tile at that location. Each tile string contains one or more tile kinds, for example "wD" specifies a wooden floor and a closed door on top of it.
- The "y" and "x" location requested, as integers (corresponding to the character's location).
- The "top", "left", "bottom", and "right" (inclusive) edges locations of the returned rectangular array, as integers.

An example INFO request may be:

```
/info?y=7&x=5
```

An example of an INFO request's JSON response with status 200 might be:

```
{
  "y": 7,
  "x": 5,
  "top": 2,
  "left": 0,
  "bottom": 12,
  "right": 10,
  "info": [
    ["S", "w", "w", "w", "w", "S", "g", "g", "g", "W", "W"],
    ["S", "w", "w", "w", "w", "S", "g", "g", "W", "W", "g"],
    ["S", "S", "S", "wD", "S", "S", "g", "g", "W", "g", "g"],
    ["g", "g", " ", " ", "g", "g", "g", "W", "W", "W", "g"],
    [".", "g", "g", "_k", "g", "g", "g", "W", "g", "W", "."],
    ["_", " ", " ", " ", "g", "g", "W", "W", "g", "W", "W"],
    ["_", "g", "t", "t", "g", "W", "W", "W", "g", "g", "W"],
    ["_", "g", "g", "t", "g", "W", "W", "g", "g", "g", "W"],
    ["_", "g", "g", "g", "g", "g", "g", "g", "g", "g", "W"],
    ["_", "g", "g", "g", "g", "g", "g", "t", "t", "g", "W", "W"],
    ["_", "g", "g", "g", "g", "g", "g", "t", "g", "g", "W", " "]
  ]
}
```


Some floor kinds block movement. For example, a brick wall prevents a character from moving into that location. Similarly, water prevents character motion.

Floor tile kinds are indicated using short strings. Each letter or symbol in a string specify a tile kind:

Letter or symbol	Tile kind	Blocks movement?
B	brick wall	blocking
b	bridge	-
D	doors closed	blocking
d	doors opened	-
-	dirt	-
f	flagstones	-
g	grass	-
p	pebbles	-
.	one rock	-
,	two rocks	-
:	three rocks	-
;	six rocks	-
S	stone wall	blocking
s	sand	-
t	tree	-
W	water waves	blocking
w	wooden boards	-

The server ensures that MOVE requests fail if that would cause the player's character to intersect a **blocking** map element.

Note that uppercase letters have been used for all **blocking** map elements, but that is a convenience, not a strict rule, and it could change in future versions of this document.

TAKE requests

Every TAKE request must be accompanied by a valid session token. For ease of explanation, this token is explained in the LOGIN request section, and will be omitted from discussion below.

The client uses a TAKE request to take an item from the map into the player character's inventory.

The character must be standing on the same location as the item, so the relative location is (0,0). Accordingly, no positional information need be sent with the request.

If an invalid TAKE request is made, the server returns a 204 status code. Requests could be invalid for a variety of reasons, the most obvious being that there is no movable item at this location.

Upon success, a TAKE request returns a 200 status. The message body may be empty, as the client doesn't consume the message body.

Items that are movable are listed in the table below:

Letter or symbol	Item kind	Item class
a	axe	tool
c	cyan potion	drink
h	heart potion	drink
k	key	artifact

Although some map elements may change state, for example doors, only the items listed above may be taken.

Each item has an item class. Items in the same class are mutually exclusive within the character's inventory. If a character issues a TAKE request for an item of a specific class, and there exists another item of the same class within that character's inventory, the two items will be swapped. For example, if a character already has a cyan potion in their inventory, and attempts to take a heart potion, the cyan potion will be placed on the ground and the heart potion will be taken into the inventory instead.

A game server may implement other rules or restrictions on items. For example, the inventory may only hold a certain maximum number of items. If the inventory is full, a TAKE request could fail with a 204, or it could succeed with a 200 but an item from the inventory might be placed on the ground as a result.

PLACE requests

Every PLACE request must be accompanied by a valid session token. For ease of explanation, this token is explained in the LOGIN request section, and will be omitted from discussion below.

The client uses a PLACE request to place an item from the character's inventory onto the ground.

The item will be placed where the character is standing, that is, at relative location is (0,0). Accordingly, no positional information need be sent with the request.

If an invalid PLACE request is made, the server returns a 204 status code. There are a number of reasons why a request may be invalid, including the character's inventory being empty.

Upon success, a PLACE request returns a 200 status. The message body may be empty, as the client doesn't consume the message body.

A game server may implement other rules or restrictions on item placement. For example, it may be possible to only place one item per map square. In such cases, a 204 response may be issued by the server if an invalid PLACE request is attempted.

USE requests

Every USE request must be accompanied by a valid session token. For ease of explanation, this token is explained in the LOGIN request section, and will be omitted from discussion below.

The client issues a USE request when the character attempts to interact with a map element that is either adjacent to or at the same location as the character.

One `dy` and one `dx` parameter may be given in each USE request. A missing parameter is assumed to be zero. So the following are valid:

Request	Validity	Notes
<code>/use?dy=-1&dx=0</code>	valid	Use element North of here.
<code>/use?dy=-1</code>	valid	Defaults <code>dx=0</code>
<code>/use?dx=+1</code>	valid	Defaults <code>dy=0</code>
<code>/use?dy=0&dx=0</code>	valid	Use element at same location as character.
<code>/use</code>	valid	Defaults <code>dy=0</code> and <code>dx=0</code> . Same as above.

But the following are invalid:

Request	Validity	Notes
<code>/move?dy=+1&dx=+1</code>	invalid	Diagonal usage is invalid.
<code>/move?dy=-2&dx=0</code>	invalid	Distance more than one space is invalid.

USE requests that are invalid produce a 204 response. Requests can be invalid because there are no usable map elements at that location, or the location is too far away.

Upon success, a USE request produces a 200 response. The message body may be empty, as the client doesn't consume the message body.

An example of a usable map element is a door. Using a door will change its state from open to closed, or from closed to open. Because the map element 'D' references a closed door, and 'd' references an open door, when a closed door is used this will have the effect of swapping a 'D' for a 'd', within the map. A subsequent INFO request will inform the client of the results of this change.

A game server may implement additional behaviour. For example, some doors may be locked, and may require the character to possess a key in order to use those doors.

Client-side View

The client displays an 11x11 grid of square locations.

Each location is drawn within the client in the following way:

1. First, the INFO request is used to discover information about that location.
2. Then each floor tile is drawn.
3. Then all items and usable map elements are drawn in order.
4. Any character at a location is then drawn over the tile.

Vision of a tile is restricted if the location is too far away (more than 5 spaces away in Y or X).

Because a tile is drawn then a character may be drawn over the top, transparent pixels in the character's sprite will reveal the pixels of the tile below.

For speed, when the player moves the character, a check for whether the tile is blocking is performed on the client-side. If the motion would succeed on the client's copy of the map, a MOVE request is made only after this check. This avoids a potentially costly round-trip to the server for every move. However, the client should undo any moves that fail as a result of 204 status codes.

The client may periodically poll the server by sending INFO requests at regular intervals in order to update the map. This is to support multiplayer interaction, and also to update the map after TAKE, PLACE, or USE requests. To help save money, it's a good idea to reduce web traffic by logging out after testing any functionality on a paid server instance.

Server-side Map

The server software must maintain a map, and serve rectangular pieces of it to clients upon request (in response to each INFO request).

A simple way to store a map is to use a text file containing the letters/symbols used for floor tile kinds. This text file can be loaded into memory when the server starts, e.g. as a rectangular array of bytes. This avoids the need for a database or anything too complex.

Because the INFO request sends each tile location as a string, locations with multiple elements are formed from strings containing multiple letters or symbols. Each letter or symbol is processed in order in order to draw elements on the client side. Thus, the "ground layer" (including elements such as grass, water, and walls) is represented by the first letter in a location string.

Multiplayer Support

To support multiple players, note the following.

There are 10 differently-coloured player avatars available for users. These use digits '0' to '9' in the map to indicate the location of each player. A good way to use these is to keep a mapping (perhaps in a text file) indicating for each username their corresponding avatar digit. This digit can then be used in the map as a tile kind when that character moves around.

Since players are drawn last, these digits should be at the end of a location string within the map. For example, the string "gk2" would indicate that at this location there is grass, a key, and then a

player character using the '2' avatar picture standing all at the same place. Player should block each other's movement, so only one avatar digit can appear in one location.

There is currently no way to assign these avatar digits from the client user interface. So this choice may be input directly on the server side.

There may be other alternative ways to assign an avatar digit rather than using a data file. For example, a hash of the username could be used to compute a number using a modulo 10 operation, however this removes choice from the user and allows multiple users to have the same avatar. There may be other techniques that could be imagined and used in experiments.

To support updates that occur as other players are interacting via the map, the client user interface regularly polls the server using INFO requests. This choice to employ polling was made to avoid introducing new (albeit possibly better) technologies such as WebSockets.

To save bandwidth, it may be valid for the server to reply to an INFO request with a 204 status code, if nothing has changed in the map since the last identical INFO request from the same client. This would be a valid server optimisation which would avoid sending multiple identical JSON objects. There may be other optimisations that could experimentally improve the server.

While testing, it may be advisable to modify the client Javascript code, to reduce the frequency of these INFO poll requests. See the `polling_time` constant value in the Javascript client.

Server-side Sessions

The server must maintain information about one or more logged in players. The session key can be used as a unique look-up key to find that information.

Information that should be maintained on the server side includes:

- Usernames.
- Encrypted passwords.
- Character icon (although this could be computed using a hash or via some other means).
- Character locations in the map.
- Inventory for each character, possibly a fixed maximum size, including whatever items are possessed by the character.

Session keys can be discarded when the user logs out, but the user information may be retained and may be restored for use when the user logs in again. Another alternative is to place all items held by a character onto the ground and then forget the user information entirely. These are game server design choices.

Document Versions

Version 1 of this document was the initial specification, including the MOVE and INFO requests.

Version 2 introduced the new LOGIN and LOGOUT requests, also adding the requirement for session tokens to be passed to all other APIs.

This is version 3 of this document. Version 3 introduces the new TAKE, PLACE, and USE requests, also allowing items, doors, multiple kinds per location string (to handle items, doors, and bridges, as well as player characters moving on the map), and full multiplayer support.